

1. WHAT IS A HEAP?

A **Heap** is a special type of **binary tree** that follows two important rules:

1. It is a **Complete Binary Tree**.
2. It follows the **Heap Property**.

A complete binary tree means that nodes are filled **level by level, from left to right**.

There are mainly two types:

- **Max Heap**
 - **Min Heap**
-

2. MAX HEAP

In a **Max Heap**, the **parent is always greater than or equal to its children**.

Example:

```
    50
   / \
  30  40
 / \  /
10 20 35
```

Check:

$50 > 30$

$50 > 40$

$30 > 10$

$30 > 20$

$40 > 35$

Therefore, it is a **Max Heap**.

Important point

The **largest element is always at the root**.

```
    50 ← Largest
   / \
  30  40
```

3. MIN HEAP

In a **Min Heap**, the **parent is always smaller than or equal to its children**.

Example:

```
    10
   / \
  20 15
 / \ /
30 40 25
```

Check:

$10 < 20$

$10 < 15$

$20 < 30$

$20 < 40$

$15 < 25$

Therefore, it is a **Min Heap**.

Important point

The **smallest element is always at the root**.

```
    10 ← Smallest
   / \
  20 15
```

4. WHAT IS A DOUBLE HEAP / DUAL HEAP?

If by "**dual heap**" you mean **two heaps used together**, it generally means maintaining:

- One **Max Heap**
- One **Min Heap**

This is commonly called a **Two-Heap** or **Dual-Heap approach**.

It is very useful for finding the **median of a set of numbers**.

Example

Suppose we have:

10, 20, 30, 40, 50

We can divide the numbers into two parts.

Max Heap — smaller half

20
/
10

Min Heap — larger half

40
/
50

And the middle value is:

30

For an odd number of elements, the median can be kept at the top of one heap or handled according to the balancing strategy.

For an even number:

10, 20, 30, 40

We can have:

Max Heap Min Heap

20 30
/
10 40

The two middle values are:

20 and 30

So:

Median = $(20 + 30) / 2$

= 25

5. SIMPLE DIFFERENCE

Heap	Root contains
Max Heap	Largest element
Min Heap	Smallest element

Dual/Two Heap Uses both Max Heap and Min Heap

Easy way to remember

MAX HEAP

Parent > Children

Root = MAXIMUM

MIN HEAP

Parent < Children

Root = MINIMUM

DUAL HEAP

Max Heap + Min Heap

Used together for problems such as Median

Real-life example

Imagine students standing according to marks.

Max Heap:

```
    95
   / \
  85  90
```

The student with **highest marks** is at the top.

Min Heap:

```
    40
   / \
  50  60
```

The student with **lowest marks** is at the top.

DUAL HEAP:

One heap manages the **lower half** of marks and another manages the **upper half**, making it efficient to find the **middle/median value**.

WHAT IS SLIDING WINDOW?

Sliding Window is a technique used to solve problems involving a **continuous portion (subarray/substring) of an array or string**.

Instead of checking every window from scratch, we **move the window step by step** and reuse the previous result.

Simple Example

Suppose we have:

Array = [2, 4, 6, 8, 10]

Window Size = 3

The windows are:

[2, 4, 6] → Window 1

[4, 6, 8] → Window 2

[6, 8, 10] → Window 3

Notice what happens:

[2, 4, 6]

↓ remove 2

[4, 6, 8]

↓ remove 4

[6, 8, 10]

The window **slides one position to the right**.

WHAT IS SLIDING WINDOW MEDIAN?

Sliding Window Median means:

Move a fixed-size window through an array and calculate the **median of the elements inside each window**.

For example:

Array = [1, 3, 5, 7, 9]

Window Size = 3

We have three windows:

[1, 3, 5]

[3, 5, 7]

[5, 7, 9]

Now calculate the median of each.

STEP 1: FIRST WINDOW

[1, 3, 5]

Elements are already sorted:

1 3 5

↑

Median

Median:

3

STEP 2: SLIDE THE WINDOW

Remove 1 and add 7.

[1, 3, 5]

↓

[3, 5, 7]

Median:

3 5 7

↑

Median = 5

STEP 3: SLIDE AGAIN

Remove 3 and add 9.

[3, 5, 7]

↓

[5, 7, 9]

Median:

5 7 9

↑

Median = 7

Final Answer

Array = [1, 3, 5, 7, 9]

Window = 3

Sliding Window Medians = [3, 5, 7]

WHAT IF WINDOW SIZE IS EVEN?

This is important.

Suppose:

Array = [1, 2, 4, 6, 8]

Window Size = 4

First window:

[1, 2, 4, 6]

There are **4 elements**, so there is no single middle element.

We take the average of the two middle elements:

1 2 4 6

↑ ↑

2 4

Therefore:

Median = $(2 + 4) / 2$

= $6 / 2$

= 3

Next window:

[2, 4, 6, 8]

Median:

$(4 + 6) / 2$

= 5

So:

Sliding Window Medians = [3, 5]

A LITTLE MORE INTERESTING EXAMPLE

Consider:

Array = [5, 2, 8, 1, 7, 3]

Window Size = 3

WINDOW 1

[5, 2, 8]

Sort:

[2, 5, 8]

Median:

5

WINDOW 2

Remove 5, add 1:

[2, 8, 1]

Sort:

[1, 2, 8]

Median:

2

WINDOW 3

Remove 2, add 7:

[8, 1, 7]

Sort:

[1, 7, 8]

Median:

7

WINDOW 4

Remove 8, add 3:

[1, 7, 3]

Sort:

[1, 3, 7]

Median:

3

Therefore:

Array = [5, 2, 8, 1, 7, 3]

K = 3

Answer = [5, 2, 7, 3]

HOW DOES TWO HEAP HELP?

For small arrays, we can simply:

Take window

↓

Sort it

↓

Find median

↓

Slide window

But sorting every window can be expensive for a large array.

For example:

Array has 1,000,000 elements

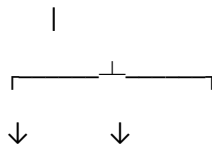
Window size = 10,000

Sorting 10,000 elements again and again is inefficient.

This is where the **Two Heap / Dual Heap technique** becomes useful.

We use:

Sliding Window



Max Heap Min Heap

Smaller half Larger half

How Two Heaps Store Data

Suppose the current window is:

[1, 3, 5, 7, 9]

We divide it into two parts.

Max Heap

Contains smaller half:

```
3
 / \
1
```

The largest element of the smaller half is at the top:

3

Min Heap

Contains larger half:

```
7
 \
9
```

The smallest element of the larger half is at the top:

7

So:

Max Heap top = 3

Min Heap top = 7

For an odd-sized window, we can keep one heap with one extra element. If the extra element is kept in the Max Heap:

Median = Max Heap top

= 5

Conceptually:

Smaller half Larger half

1 3 5 | 7 9

↑

Median

Even Window

Suppose:

[1, 3, 5, 7]

Divide:

Smaller half Larger half

1 3 | 5 7

↑ ↑

Max Min

Heap Heap

Therefore:

Median = (3 + 5) / 2

= 4

So for an even window:

Median = (MaxHeap.top + MinHeap.top) / 2

Sliding Window Median Process

Suppose:

Array = [1, 3, -1, -3, 5, 3, 6, 7]

K = 3

Windows:

[1, 3, -1]

[3, -1, -3]

[-1, -3, 5]

[-3, 5, 3]

[5, 3, 6]

[3, 6, 7]

Calculate median:

[1, 3, -1]

Sorted = [-1, 1, 3]

Median = 1

[3, -1, -3]

Sorted = [-3, -1, 3]

Median = -1

[-1, -3, 5]

Sorted = [-3, -1, 5]

Median = -1

[-3, 5, 3]

Sorted = [-3, 3, 5]

Median = 3

[5, 3, 6]

Sorted = [3, 5, 6]

Median = 5

[3, 6, 7]

Sorted = [3, 6, 7]

Median = 6

Final result

[1, -1, -1, 3, 5, 6]

Easy Formula to Remember

Odd window

If:

$K = 5$

After sorting:

[a, b, c, d, e]

Median:

c

Index:

$K // 2$

Even window

If:

$K =$

After sorting:

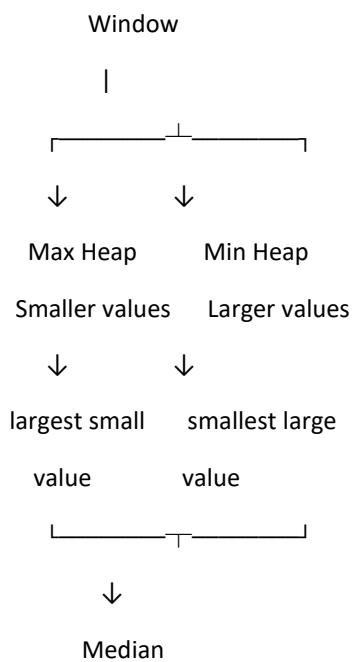
[a, b, c, d]

Median:

$(b + c) / 2$

Why Two Heaps?

The main idea is:



When the window slides:

1. **Remove** the element leaving the window.

2. **Add** the new element.
3. **Balance** the two heaps.
4. Read the median from the heap tops.